

Pruebas unitarias con NUnit

para realizar pruebas unitarias en C# trabajando con NUnit que necesito?

Si vas a trabajar con **NUnit** en C#, necesitás muy pocas cosas. Si ya tenés instalado el SDK de .NET, en unos minutos podés empezar a escribir pruebas.

1. Tener instalado el SDK de .NET

Verificá desde la consola:

```
</> Bash  
dotnet --version
```

Si devuelve una versión (por ejemplo `9.0.xxx` o `8.0.xxx`), ya está instalado.

2. Tener el proyecto a probar

Por ejemplo:

```
Calculadora/  
|  
├─ Calculadora.csproj  
├─ Program.cs  
└─ Calculadora.cs
```

3. Crear un proyecto de pruebas

Desde la carpeta donde está la solución:

```
</> Bash  
dotnet new nunit -n Calculadora.Tests
```

Se crea:

```
Calculadora.Tests/  
|  
├─ Calculadora.Tests.csproj  
└─ UnitTest1.cs
```

4. Agregar la referencia al proyecto principal

Entrar al proyecto de pruebas:

```
</> Bash  
cd Calculadora.Tests
```

y ejecutar:

```
</> Bash  
dotnet add reference ../Calculadora/Calculadora.csproj
```

Así las pruebas podrán usar las clases del proyecto.

5. Los paquetes que utiliza NUnit

Generalmente el template ya instala automáticamente:

- NUnit
- NUnit3TestAdapter
- Microsoft.NET.Test.Sdk

Si querés verificar:

```
</> Bash  
dotnet list package
```

Deberías ver algo parecido a:

```
Microsoft.NET.Test.Sdk  
NUnit  
NUnit3TestAdapter  
coverlet.collector
```

6. Escribir una prueba

Supongamos esta clase:

```
</> C#  
public class Calculadora  
{  
    public int Sumar(int a, int b)  
    {  
        return a + b;  
    }  
}
```

```
}  
}
```

La prueba:

</> C#

```
using NUnit.Framework;  
  
namespace Calculadora.Tests;  
  
public class CalculadoraTest  
{  
    [Test]  
    public void Sumar_DosNumeros_RetornaResultadoCorrecto()  
    {  
        // Arrange  
        Calculadora calc = new Calculadora();  
  
        // Act  
        int resultado = calc.Sumar(3, 5);  
  
        // Assert  
        Assert.That(resultado, Is.EqualTo(8));  
    }  
}
```

7. Ejecutar las pruebas

Desde la carpeta del proyecto de pruebas:

</> Bash

```
dotnet test
```

Obtendrás algo similar a:

```
Passed! - Failed: 0  
Passed: 1  
Skipped: 0  
Total: 1
```

Estructura típica

```
MiSolucion  
|  
├─ Calculadora
```

```
| |─ Calculadora.csproj
| |─ Calculadora.cs
|
└─ Calculadora.Tests
    │─ Calculadora.Tests.csproj
    └─ CalculadoraTest.cs
```

Atributos más utilizados de NUnit

Atributo	Función
[Test]	Indica un método de prueba.
[SetUp]	Se ejecuta antes de cada prueba.
[TearDown]	Se ejecuta después de cada prueba.
[OneTimeSetUp]	Se ejecuta una sola vez antes de todas las pruebas.
[OneTimeTearDown]	Se ejecuta una sola vez al finalizar todas las pruebas.
[TestCase]	Ejecuta el mismo método con distintos parámetros.
[Ignore]	Omite una prueba.

Ejemplo con varios casos:

<> C#

```
[TestCase(2, 3, 5)]
[TestCase(10, 5, 15)]
[TestCase(-1, 1, 0)]
public void Sumar(int a, int b, int esperado)
{
    Calculadora calc = new Calculadora();

    int resultado = calc.Sumar(a, b);

    Assert.That(resultado, Is.EqualTo(esperado));
}
```

Assertions más comunes

</> C#

```
Assert.That(valor, Is.EqualTo(10));

Assert.That(valor, Is.Not.EqualTo(5));

Assert.That(objeto, Is.Null);

Assert.That(objeto, Is.Not.Null);

Assert.That(lista.Count, Is.EqualTo(5));

Assert.That(numero, Is.GreaterThan(10));

Assert.That(numero, Is.LessThan(20));

Assert.That(texto, Does.Contain("Hola"));

Assert.That(texto, Does.StartsWith("Ho"));

Assert.That(texto, Does.EndsWith("la"));

Assert.Throws<DivideByZeroException>(() =>
{
    calculadora.Dividir(10, 0);
});
```

Si trabajás con Visual Studio Code

Solo necesitás:

- .NET SDK instalado.
- La extensión **C# Dev Kit** (o la extensión oficial de C# de Microsoft).
- El proyecto de pruebas creado con `dotnet new nunit`.

Luego podés ejecutar las pruebas desde la terminal con `dotnet test` o utilizar el **Testing Explorer** integrado de VS Code para ejecutarlas y depurarlas gráficamente.

Para un curso de Programación Orientada a Objetos, NUnit es una excelente elección porque permite enseñar pruebas unitarias siguiendo el patrón **Arrange – Act – Assert (AAA)** de forma muy clara y gradual.



Exactamente. En el ecosistema Java, el equivalente más utilizado de **NUnit** es **JUnit 5 (Jupiter)**. La idea es prácticamente la misma:

C#	Java
NUnit	JUnit 5
[Test]	@Test
Assert.That()	assertEquals() , assertTrue() , etc.
dotnet test	mvn test

Como ya venís trabajando con **Maven** y **VS Code**, te propongo hacerlo igual que el ejemplo de C#, para que los alumnos vean las similitudes.

Práctica: Pruebas Unitarias con JUnit 5

Objetivo

Crear una calculadora sencilla y verificar su funcionamiento mediante dos pruebas unitarias utilizando **JUnit 5**.

Paso 1. Crear el proyecto Maven

Abrir VS Code.

Abrir una terminal y ejecutar:

```
</> Bash
```

```
mvn archetype:generate
```

Elegir:

```
org.apache.maven.archetypes:maven-archetype-quickstart
```

Completar:

```
GroupId: ar.edu.universidad  
ArtifactId: CalculadoraJUnit  
Version: 1.0-SNAPSHOT
```

Ingresar al proyecto:

```
</> Bash
```

```
cd CalculadoraJUnit
```

Paso 2. Observar la estructura

Maven crea automáticamente:

```
CalculadoraJUnit
|
|-- pom.xml
|
|-- src
|   |-- main
|   |   |-- java
|   |   |   |-- ar
|   |   |   |   |-- edu
|   |   |   |   |   |-- universidad
|   |   |   |   |   |   |-- App.java
|   |
|   |-- test
|   |   |-- java
|   |   |   |-- ar
|   |   |   |   |-- edu
|   |   |   |   |   |-- universidad
|   |   |   |   |   |   |-- AppTest.java
```

Observación para los alumnos:

- **main** → código de la aplicación.
- **test** → pruebas unitarias.

Paso 3. Verificar el POM

Debe incluir JUnit 5.

</> XML

```
<dependencies>

  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <version>5.13.4</version>
    <scope>test</scope>
  </dependency>

</dependencies>
```

Y el plugin:

</> XML

```
<build>

  <plugins>

    <plugin>

      <groupId>org.apache.maven.plugins</groupId>

      <artifactId>maven-surefire-plugin</artifactId>

      <version>3.5.4</version>

    </plugin>

  </plugins>

</build>
```

Paso 4. Crear la clase Calculadora

Dentro de

src/main/java/ar/edu/universidad

crear

</> Java

```
package ar.edu.universidad;

public class Calculadora {

    public int sumar(int a, int b) {
        return a + b;
    }

    public int restar(int a, int b) {
        return a - b;
    }

}
```

Paso 5. Modificar App.java

</> Java


```
package ar.edu.universidad;

public class App {

    public static void main(String[] args) {

        Calculadora calculadora = new Calculadora();

        System.out.println("5 + 3 = " + calculadora.sumar(5,3));

        System.out.println("10 - 4 = " + calculadora.restar(10,4));

    }

}
```

Ejecutar:

 **Bash**

```
mvn compile
```

o desde VS Code.

Resultado:

```
5 + 3 = 8
10 - 4 = 6
```

Paso 6. Crear la clase de prueba

En

```
src/test/java/ar/edu/universidad
```

crear

```
CalculadoraTest.java
```

Paso 7. Escribir las pruebas

 **Java**

```
package ar.edu.universidad;

import org.junit.jupiter.api.Test;
```

```
import static org.junit.jupiter.api.Assertions.assertEquals;

public class CalculadoraTest {

    @Test
    public void pruebaSuma() {

        // Arrange
        Calculadora calculadora = new Calculadora();

        // Act
        int resultado = calculadora.sumar(4,3);

        // Assert
        assertEquals(7, resultado);

    }

    @Test
    public void pruebaResta() {

        // Arrange
        Calculadora calculadora = new Calculadora();

        // Act
        int resultado = calculadora.restar(10,4);

        // Assert
        assertEquals(6, resultado);

    }

}
```

Paso 8. Ejecutar las pruebas

Desde la terminal:

```
</> Bash
```

```
mvn test
```

Resultado:

```
Tests run: 2
```

```
Failures: 0
```

```
Errors: 0
```

BUILD SUCCESS

Paso 9. Comprobar un error

Modificar:

</> Java

```
assertEquals(10, resultado);
```

Ejecutar nuevamente:

</> Bash

```
mvn test
```

Resultado:

Tests run: 2

Failures: 1

Expected :10

Actual :7

Así se observa cómo JUnit informa exactamente qué prueba falló y por qué.

Luego volver a:

</> Java

```
assertEquals(7, resultado);
```

Estructura final

CalculadoraJUnit

```
|
|— pom.xml
|
|— src
|   |— main
|   |   |— java
|   |   |   |— ar
|   |   |   |   |— edu
|   |   |   |   |   |— universidad
```

```
|
|
|
|
└─ test
    └─ java
        └─ ar
            └─ edu
                └─ universidad
                    └─ CalculadoraTest.java
```

```
└─ App.java
└─ Calculadora.java
```

Equivalencias entre JUnit y NUnit

Concepto	JUnit 5 (Java)	NUnit (C#)
Marcar una prueba	<code>@Test</code>	<code>[Test]</code>
Comparar resultados	<code>assertEquals(esperado, obtenido)</code>	<code>Assert.That(obtenido, Is.EqualTo(esperado))</code>
Preparación antes de cada prueba	<code>@BeforeEach</code>	<code>[SetUp]</code>
Limpieza después de cada prueba	<code>@AfterEach</code>	<code>[TearDown]</code>
Ejecutar pruebas	<code>mvn test</code>	<code>dotnet test</code>

Comparación didáctica

Este ejemplo es ideal para mostrar que el concepto de **prueba unitaria** es el mismo en distintos lenguajes:

- En **Java** se utiliza **JUnit 5**.
- En **C#** se utiliza **NUnit** (aunque también existen MSTest y xUnit).
- Ambos frameworks siguen el patrón **AAA (Arrange – Act – Assert)**.
- Ambos permiten automatizar la verificación del código, detectar errores rápidamente y ejecutar todas las pruebas con un solo comando (`mvn test` o `dotnet test`).



modificame el groupId como com.educacionIT

Sí. Si querés usar el mismo convenio que venís utilizando en tus ejemplos, el **GroupId** quedaría:

```
GroupId: com.educacionIT
ArtifactId: CalculadoraJUnit
Version: 1.0-SNAPSHOT
```

En consecuencia, los paquetes Java pasan a ser:

```
</> Java
package com.educacionIT;
```

La estructura del proyecto quedaría:

```
CalculadoraJUnit
|
|-- pom.xml
|
|-- src
|   |-- main
|   |   |-- java
|   |   |   |-- com
|   |   |   |   |-- educacionIT
|   |   |   |   |   |-- App.java
|   |   |   |   |   |-- Calculadora.java
|   |   |
|   |   |-- test
|   |   |   |-- java
|   |   |   |   |-- com
|   |   |   |   |   |-- educacionIT
|   |   |   |   |   |-- CalculadoraTest.java
```

Y las clases comenzarían así:

Calculadora.java

```
</> Java
package com.educacionIT;

public class Calculadora {

    public int sumar(int a, int b) {
        return a + b;
    }

    public int restar(int a, int b) {
        return a - b;
    }
}
```

App.java

Java

```
package com.educacionIT;

public class App {

    public static void main(String[] args) {

        Calculadora calculadora = new Calculadora();

        System.out.println("5 + 3 = " + calculadora.sumar(5, 3));
        System.out.println("10 - 4 = " + calculadora.restar(10, 4));

    }

}
```

CalculadoraTest.java

Java

```
package com.educacionIT;

import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;

public class CalculadoraTest {

    @Test
    public void pruebaSuma() {

        // Arrange
        Calculadora calculadora = new Calculadora();

        // Act
        int resultado = calculadora.sumar(4, 3);

        // Assert
        assertEquals(7, resultado);
    }

    @Test
    public void pruebaResta() {

        // Arrange
        Calculadora calculadora = new Calculadora();

        // Act
        int resultado = calculadora.restar(10, 4);

        // Assert
    }
```

```
        assertEquals(6, resultado);  
    }  
}
```

De esta manera, todo el ejemplo queda consistente con el paquete `com.educacionIT`, que es una convención muy utilizada en proyectos Java para fines didácticos.



sigue el mismo ejemplo en python y luego en JS con el paso a paso

Me parece una excelente idea. Si los alumnos ven exactamente el mismo ejercicio en **C#**, **Java**, **Python** y **JavaScript**, entienden que **las pruebas unitarias son un concepto, no algo propio de un lenguaje**.

PRÁCTICA 3 - PYTHON + pytest

Objetivo

Crear una calculadora sencilla y verificar su funcionamiento mediante **dos pruebas unitarias** utilizando **pytest**.

Paso 1. Crear la carpeta del proyecto

```
CalculadoraPytest
```

Abrir la carpeta con VS Code.

Paso 2. Crear un entorno virtual (opcional pero recomendado)

Windows

```
<> Bash  
python -m venv .venv
```

Activar

```
<> Bash
```

```
.venv\Scripts\activate
```

Paso 3. Instalar pytest

❏ Bash

```
pip install pytest
```

Comprobar

❏ Bash

```
pytest --version
```

Debe aparecer algo similar a

```
pytest 8.x.x
```

Paso 4. Crear la calculadora

Crear

```
calculadora.py
```

❏ Python

```
class Calculadora:

    def sumar(self, a, b):
        return a + b

    def restar(self, a, b):
        return a - b
```

Paso 5. Probar manualmente

Crear

```
main.py
```

❏ Python


```
from calculadora import Calculadora

calculadora = Calculadora()

print("5 + 3 =", calculadora.sumar(5,3))
print("10 - 4 =", calculadora.restar(10,4))
```

Ejecutar

🔗 Bash

```
python main.py
```

Resultado

```
5 + 3 = 8
10 - 4 = 6
```

Paso 6. Crear las pruebas

Crear

```
test_calculadora.py
```

🔗 Python

```
from calculadora import Calculadora

def test_suma():

    # Arrange
    calculadora = Calculadora()

    # Act
    resultado = calculadora.sumar(4,3)

    # Assert
    assert resultado == 7

def test_resta():

    # Arrange
    calculadora = Calculadora()

    # Act
    resultado = calculadora.restar(10,4)
```

```
# Assert  
assert resultado == 6
```

Paso 7. Ejecutar las pruebas

❏ Bash

```
pytest
```

Resultado

```
=====
```

2 passed

```
=====
```

Paso 8. Provocar un error

Modificar

❏ Python

```
assert resultado == 10
```

Ejecutar nuevamente

❏ Bash

```
pytest
```

Resultado

```
FAILED test_suma
```

Expected: 10

Actual: 7

Volver a dejar

❏ Python

```
assert resultado == 7
```

Estructura final

```
CalculadoraPytest
|
├── calculadora.py
├── main.py
└── test_calculadora.py
```

PRÁCTICA 4 - JAVASCRIPT + JEST

Objetivo

Crear la misma calculadora y verificar su funcionamiento mediante **Jest**.

Paso 1. Crear el proyecto

```
CalculadoraJest
```

Abrir con VS Code.

Paso 2. Inicializar Node

```
</> Bash
npm init -y
```

Se crea

```
package.json
```

Paso 3. Instalar Jest

```
</> Bash
npm install --save-dev jest
```

Paso 4. Modificar package.json

Cambiar

JSON

```
"scripts": {  
  "test": "jest"  
}
```

Paso 5. Crear la calculadora

calculadora.js

JavaScript

```
class Calculadora {  
  
  sumar(a,b){  
    return a+b;  
  }  
  
  restar(a,b){  
    return a-b;  
  }  
  
}  
  
module.exports = Calculadora;
```

Paso 6. Probar manualmente

Crear

app.js

JavaScript

```
const Calculadora = require("./calculadora");  
  
const calculadora = new Calculadora();  
  
console.log("5 + 3 =", calculadora.sumar(5,3));  
  
console.log("10 - 4 =", calculadora.restar(10,4));
```

Ejecutar

</> Bash`node app.js`

Resultado

```
5 + 3 = 8
10 - 4 = 6
```

Paso 7. Crear la prueba

`calculadora.test.js`**</> JavaScript**

```
const Calculadora = require("./calculadora");

test("Prueba de suma", ()=>{

    // Arrange
    const calculadora = new Calculadora();

    // Act
    const resultado = calculadora.sumar(4,3);

    // Assert
    expect(resultado).toBe(7);

});

test("Prueba de resta", ()=>{

    // Arrange
    const calculadora = new Calculadora();

    // Act
    const resultado = calculadora.restar(10,4);

    // Assert
    expect(resultado).toBe(6);

});
```

Paso 8. Ejecutar las pruebas

```
</> Bash
```

```
npm test
```

Resultado

PASS

2 tests passed

Paso 9. Provocar un error

Modificar

```
</> JavaScript
```

```
expect(resultado).toBe(10);
```

Ejecutar nuevamente

```
</> Bash
```

```
npm test
```

Resultado

FAIL

Expected: 10

Received: 7

Luego volver a

```
</> JavaScript
```

```
expect(resultado).toBe(7);
```

Estructura final

```
CalculadoraJest
```

```
|
```

```
├─ package.json
```

```
├─ calculadora.js
```

```
├─ app.js
```

```
└─ calculadora.test.js
```

Resumen comparativo

Lenguaje	Framework	Ejecutar aplicación	Ejecutar pruebas	Anotación/Función de prueba	Verificación
C#	NUnit	<code>dotnet run</code>	<code>dotnet test</code>	<code>[Test]</code>	<code>Assert.That()</code>
Java	JUnit 5	<code>mvn</code> <code>exec:java</code> o desde VS Code	<code>mvn test</code>	<code>@Test</code>	<code>assertEquals()</code>
Python	pytest	<code>python</code> <code>main.py</code>	<code>pytest</code>	<code>def test_...()</code>	<code>assert</code>
JavaScript	Jest	<code>node</code> <code>app.js</code>	<code>npm test</code>	<code>test()</code>	<code>expect(...).to</code>

Conclusión para los alumnos

Aunque cambie el lenguaje, la lógica de una prueba unitaria es siempre la misma:

- 1. Arrange:** preparar los datos y crear los objetos necesarios.
- 2. Act:** ejecutar el método que se desea probar.
- 3. Assert:** verificar que el resultado obtenido coincida con el esperado.

La principal diferencia entre los lenguajes está en la sintaxis y en el framework utilizado (JUnit, NUnit, pytest o Jest), pero el proceso de validar el comportamiento del código es esencialmente idéntico.

